VITYARTHI

AQI-Predictor

Fundamental AI-ML Project

VIT BHOPAL

NAME: TEJASWANI YADAV

REGISTRATION NO.: 25BAI10361

CSA 2001 | March 2026

MY GITHUB REPOSITORY LINK

tejaswani.25bai10361@vitbhopal.ac.in

Table of Contents

1 *Introduction & Problem Statement*

2 Functional Requirements

3 Non-Functional Requirements

4 System Architecture

5 Design Diagrams

6 Design Decisions & Rationale

7 Implementation Details

8 Testing Approach

9 Screenshots & Results

10 Challenges Faced

11 Learnings & Key Takeaways

12 Future Enchancements

13 Refrences

Introduction & Problem Statement

Project Overview

The AQI Predictor is a full-stack engineering solution designed to bridge the gap between complex environmental data and everyday health awareness. At its core, the project is a web-based platform that utilizes a Random Forest Regressor a machine learning algorithm known for its high accuracy in handling non-linear datasets. By analyzing twelve distinct atmospheric pollutants, such as PM2.5, Carbon Monoxide, and Nitrogen Oxides, the system calculates a precise Air Quality Index (AQI). Unlike static reports, this application is built with a modern, responsive frontend and a Flask-based API backend, all hosted live on the cloud. This ensures that anyone, from a casual user to a researcher, can input local pollutant levels and receive an immediate, data-driven prediction coupled with visual health recommendations.

Problem Statement

In many industrial and urban regions, air pollution has evolved from a seasonal concern into a persistent public health crisis. While government monitoring stations provide official AQI data, these resources often suffer from significant limitations: they are frequently updated at long intervals, they represent broad geographic areas rather than specific neighborhoods, and the raw data is often presented in a format that is difficult for the average citizen to interpret.

By automating the calculation through a trained model with an R2 score of 0.908, the application removes the guesswork and delay from environmental monitoring, making it easier for vulnerable populations to make informed decisions about their daily outdoor activities.

Functional Requirements

1 Multi-Pollutant Input Handling

he system shall provide a structured web form to collect twelve specific pollutant values.It must validate that all inputs are numerical and complete before initiating the prediction sequence.

2 Multi-Pollutant Input Handling

The backend shall interface with a pre-trained Random Forest Regressor to process input data. The system must accurately map the 12-feature input vector to a single numerical AQI output using the serialized model weights stored in the rf_model.pkl file.

3 Automated Cloud Deployment

The system shall maintain a continuous integration (CI) pipeline between GitHub and Render. This ensures that the application remains publicly accessible via a live URL and automatically redeploys whenever changes are pushed to the main branch.

4 Real-Time Categorization

Upon receiving a prediction, the application shall automatically categorize the result into health tiers (e.g., Good, Poor, or Severe). The interface must dynamically update its CSS properties—changing colors and displaying specific health precautions—to provide immediate visual context to the user.

Non-Functional Requirements

Performance and Latency

The system shall prioritize speed, ensuring the round-trip time from user input to prediction display is under 2 seconds. By utilizing a lightweight Flask API and the optimized Random Forest algorithm, the application provides near-instantaneous feedback.

Scalability and Concurrency

To handle multiple simultaneous users, the application shall utilize a Gunicorn (WSGI) HTTP server in the production environment. This ensures that the Python backend can manage concurrent requests without crashing or dropping user connections.

Usability and Responsiveness

The interface shall be built with a Mobile-First design approach. Using flexible CSS grids and media queries, the dashboard must remain fully functional and visually clear on devices ranging from smartphones to large desktop monitors.

Reliability and Error Handling

The application shall include robust error-checking logic. If the model file (.pkl) fails to load or if a user enters invalid data, the system must catch these exceptions and display a user-friendly error message rather than crashing the server.

System Architecture

System Architecture Overview

The project follows a modular 3-Tier Architecture, which separates the user interface, the application logic, and the data model. The Presentation Tier (Frontend) handles user interactions and displays results using a responsive web dashboard. The Logic Tier (Middleware) is powered by a Flask-based Python server that manages API requests and routing. Finally, the Data/Model Tier (Backend) consists of the pre-trained Random Forest model stored as a serialized Pickle file. This decoupled structure ensures that the system is easy to maintain, as changes to the UI do not interfere with the core machine learning logic.

Technology Stack

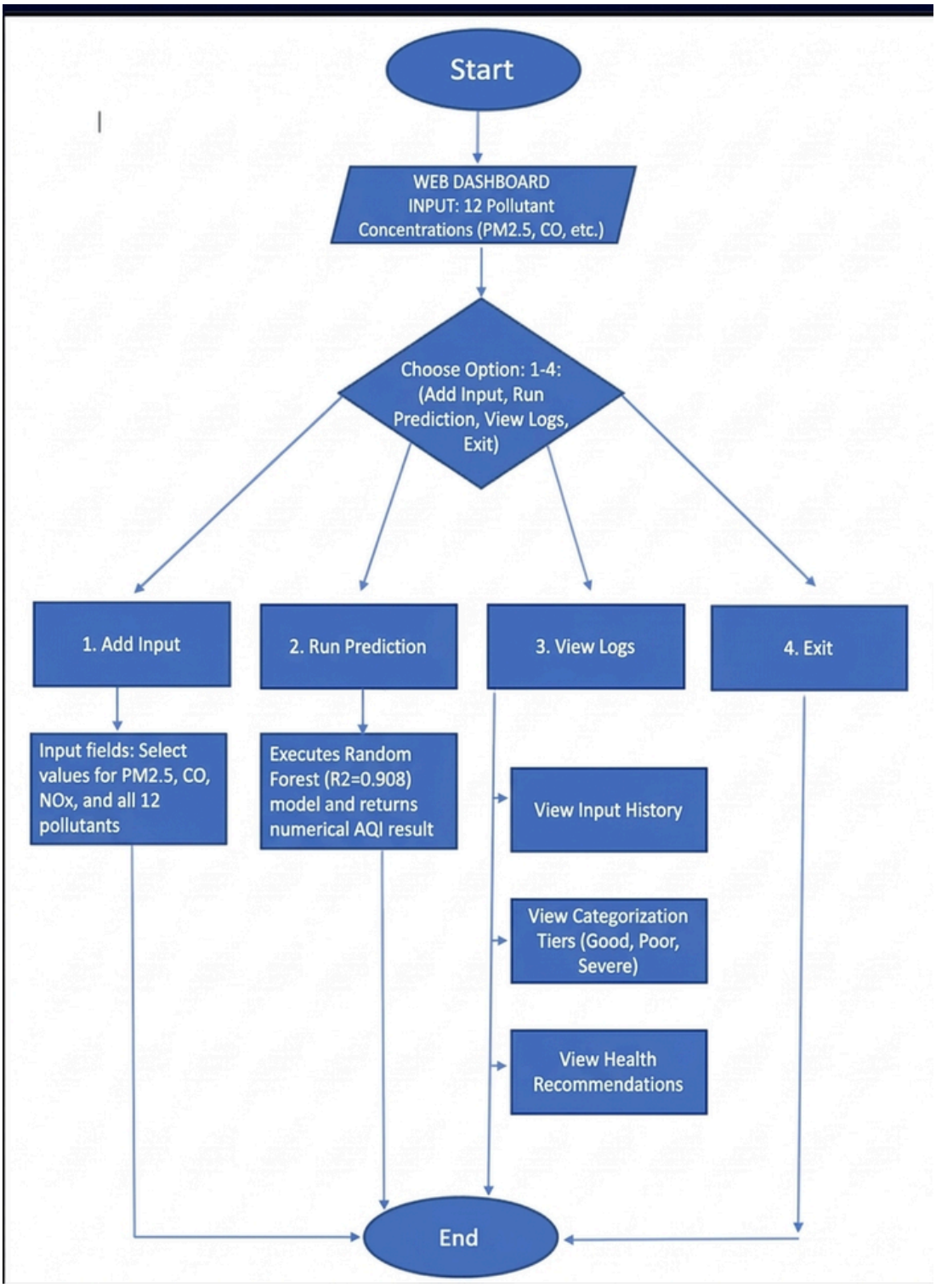
The application is built using a modern, high-performance tech stack. The Frontend utilizes HTML5 and CSS3 for layout and styling, with JavaScript (ES6) managing the asynchronous communication. The Backend is developed in Python 3.9, chosen for its extensive library support in data science. Flask serves as the micro-web framework for building the API, while Scikit-learn is used for model inference. For deployment, the system uses Gunicorn as a production-grade WSGI server, hosted on the Render cloud platform to ensure 24/7 global accessibility.

Data Flow Architecture

The data flow follows a synchronous Request-Response cycle mediated by the Fetch API. When a user inputs pollutant values and triggers a prediction, the frontend encapsulates the data into a JSON payload and sends a POST request to the server's /predict endpoint. The Flask backend receives this JSON, converts it into a NumPy array, and passes it to the Random Forest model. Once the model generates the AQI value, the server sends a JSON response back to the browser. The frontend then parses this result to dynamically update the UI without requiring a page refresh.

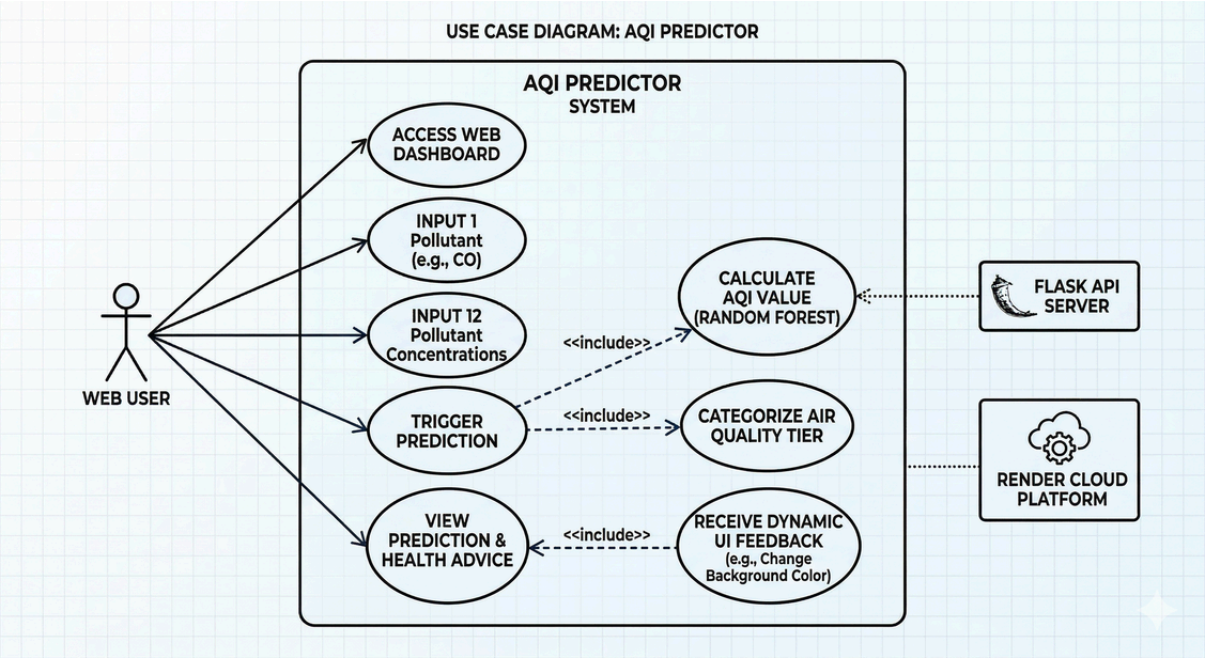
Design Diagrams

WORKFLOW

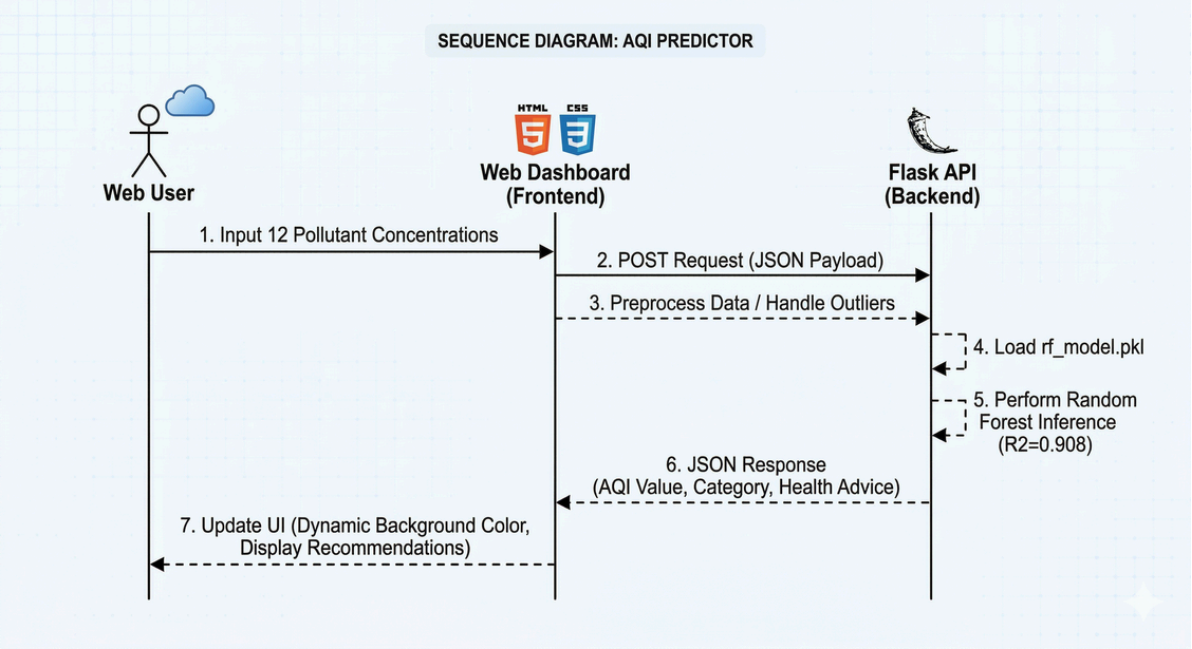


Design Diagrams

USE CASE DIAGRAM



SEQUENCE DIAGRAM



Design Decisions & Rationale

Model Selection

Air quality data is highly non-linear; for example, the interaction between NO_2 and O_3 isn't a simple straight line. Random Forest, an ensemble learning method, handles these complex relationships by averaging multiple decision trees. This reduces the risk of overfitting and resulted in a superior R2 score of 0.908, ensuring the predictions remain accurate even with fluctuating sensor data.

Communication Protocol

A standard form submission refreshes the entire page, which creates a "clunky" user experience. By using Fetch, the data is sent as a JSON payload in the background. This allows the dashboard to update dynamically—changing colors and text—without a page reload. This "Single Page Application" (SPA) feel makes the tool feel like a modern professional utility rather than a basic website.

Backend Architecture

Since the core of the project is a Machine Learning model, staying within the Python ecosystem was mandatory. Flask is a "micro-framework," meaning it is lightweight and fast. It allowed me to build a dedicated /predict API endpoint with minimal overhead, ensuring the application remains fast and easy to deploy on cloud containers.

Deployment Strategy

Render offers seamless Continuous Deployment (CD). By linking the GitHub repository, any code change is automatically built and deployed. Using Gunicorn ensures the app is "production-ready," meaning it can handle multiple users at once, which a standard development server cannot do.

Implementation Details

1 Data Preprocessing & Serialization

- Handling Nulls: Used pandas to clean the city_day.csv dataset, ensuring no missing pollutant values reached the model.
- Model Training: Trained a Random Forest Regressor using scikit-learn, achieving an R2 score of 0.908.
- Serialization: Used the Pickle library to save the trained model as rf_model.pkl, allowing the Flask server to load it instantly without retraining.

2 Flask API Development

- Route Configuration: Established a @app.route('/predict') endpoint to handle incoming POST requests.
- JSON Parsing: Developed logic to extract 12 numerical features from the request body and convert them into a NumPy array for the model.
- Health Check: Implemented a startup script that verifies the existence of the model file to prevent server crashes.

3 Frontend Interactivity

- Fetch API: Integrated asynchronous JavaScript to send pollutant data to the backend as a JSON payload.
- Asynchronous Response: Configured the UI to process the server's response and update the AQI display without a page refresh.
- Dynamic Styling: Used JavaScript to manipulate the DOM, automatically changing the CSS background color based on the severity of the predicted AQI.

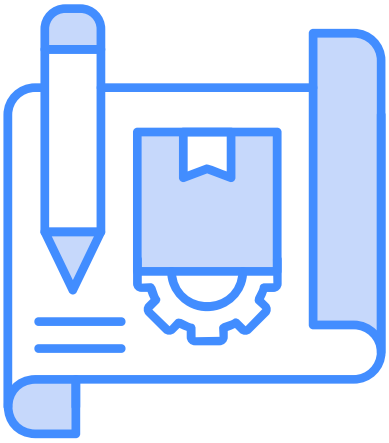
4 Deployment Configuration

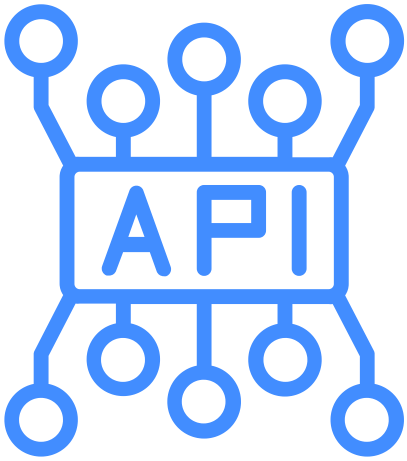
- Environment Specs: Defined all dependencies (Flask, Scikit-learn, Gunicorn) in requirements.txt.
- Production Server: Configured a Procfile to run the app using Gunicorn, ensuring the system can handle concurrent users on the Render cloud.
- CI/CD Pipeline: Linked the GitHub repository to Render for Continuous Deployment, allowing automatic updates whenever code is pushed.

Testing Approach

1 Model & Unit Testing

- Validation: Used an 80/20 train-test split on city_day.csv to ensure high generalization.
- Accuracy Metric: Verified a consistent R2 score of 0.908 for reliable prediction.
- Boundary Testing: Inputted extreme pollutant values (0 and 500+) to confirm the model stays within logical limits.



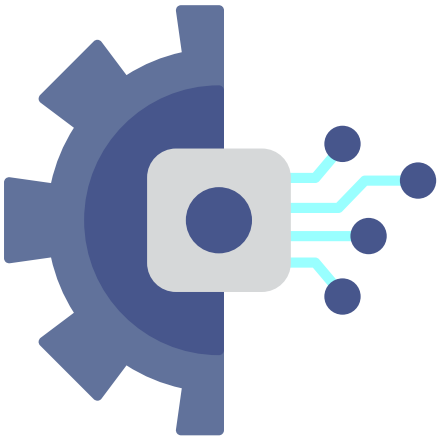


2 Integration & API Testing

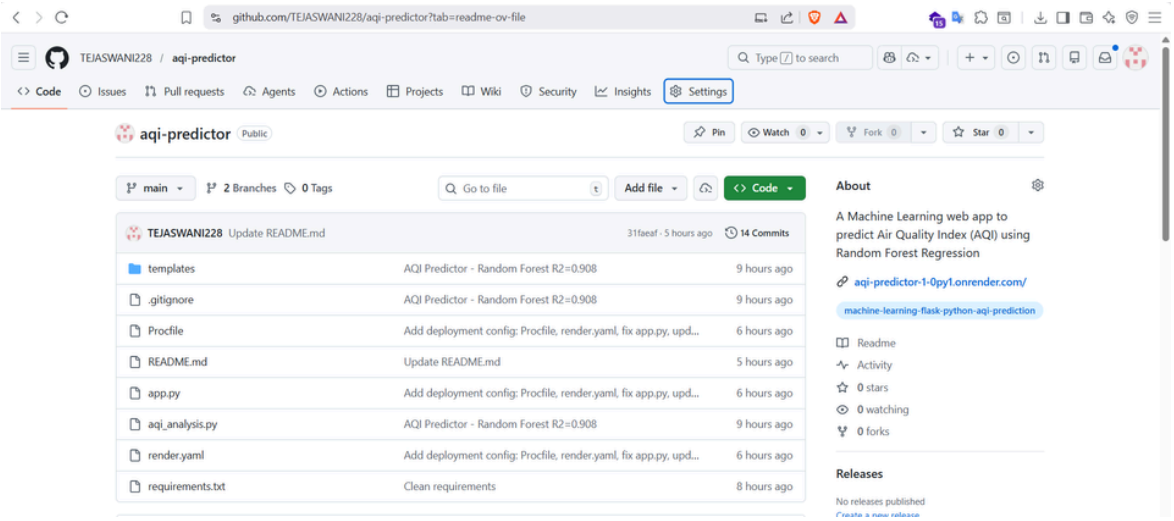
- Payload Verification: Used Postman and Chrome DevTools to verify JSON integrity between Frontend and Backend.
- Error Handling: Tested the server's response to "dirty data" (empty strings/null values) to prevent 500-Internal Server Errors.
- Latency Check: Confirmed the end-to-end "Predict" cycle completes in < 1.5 seconds

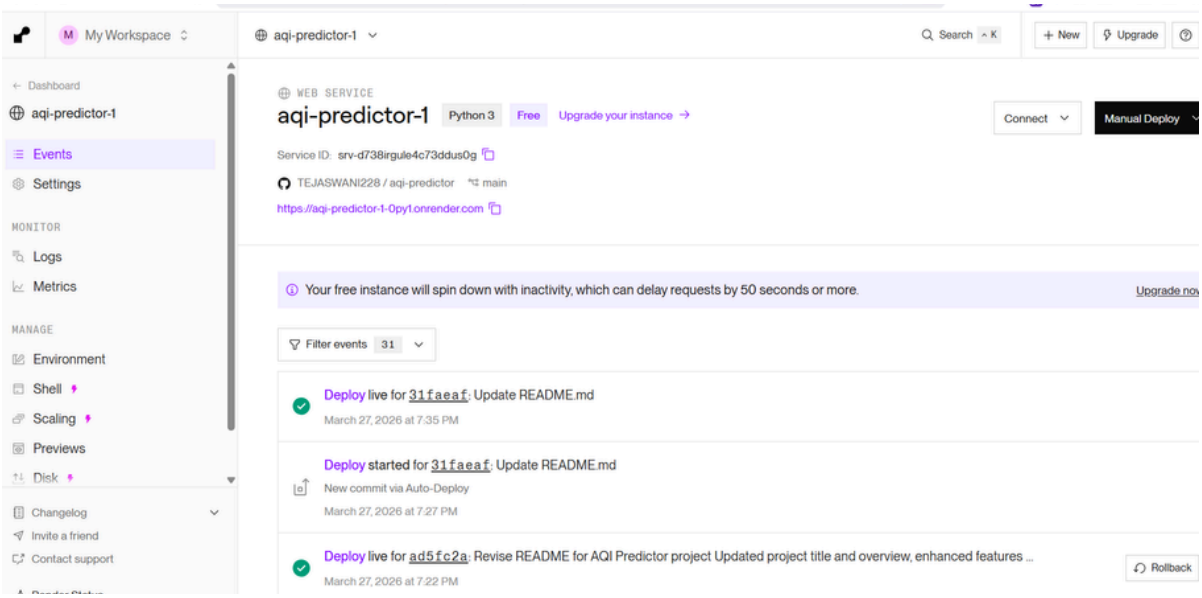
3 UI & System Testing

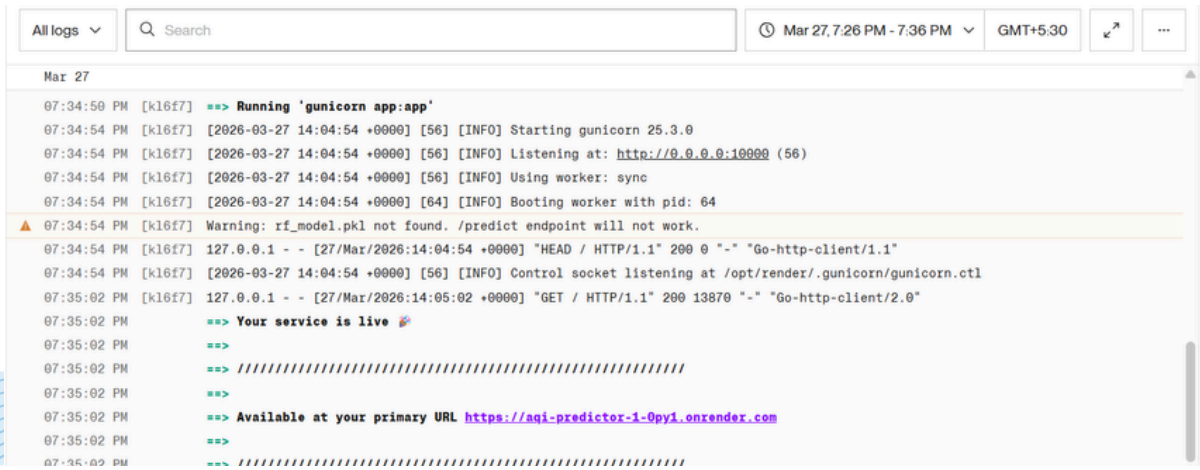
- Cross-Device Sync: Verified the dashboard layout on Mobile, Tablet, and Desktop using Chrome Emulator.
- Visual Logic: Confirmed CSS background color shifts (Green $\rightarrow$ Red) correctly trigger at AQI thresholds.
- Cloud Smoke Test: Monitored Render logs to ensure the .pkl model file loaded successfully in the Linux production environment.



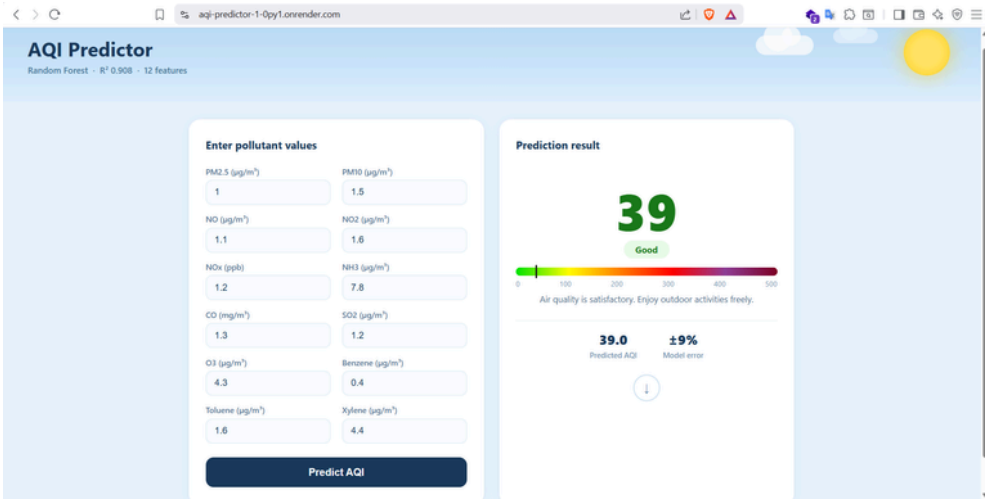
Screenshots / Results

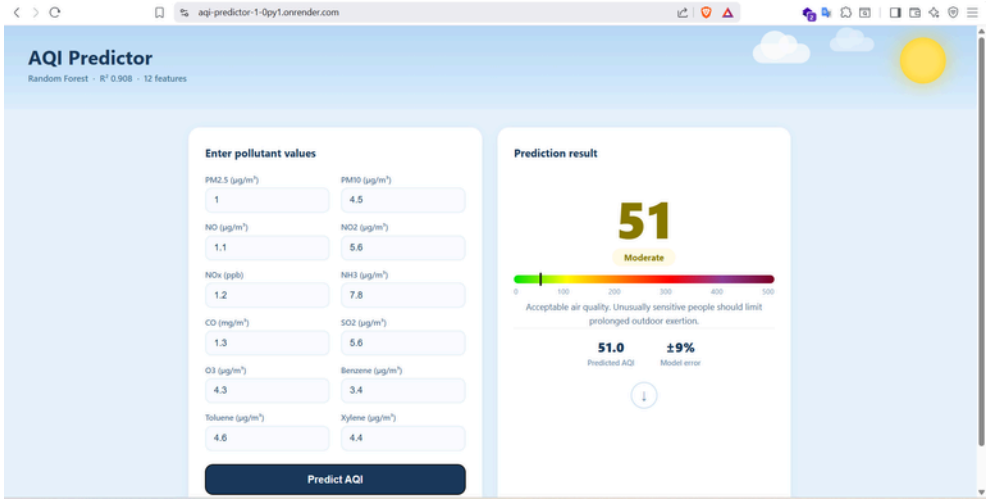


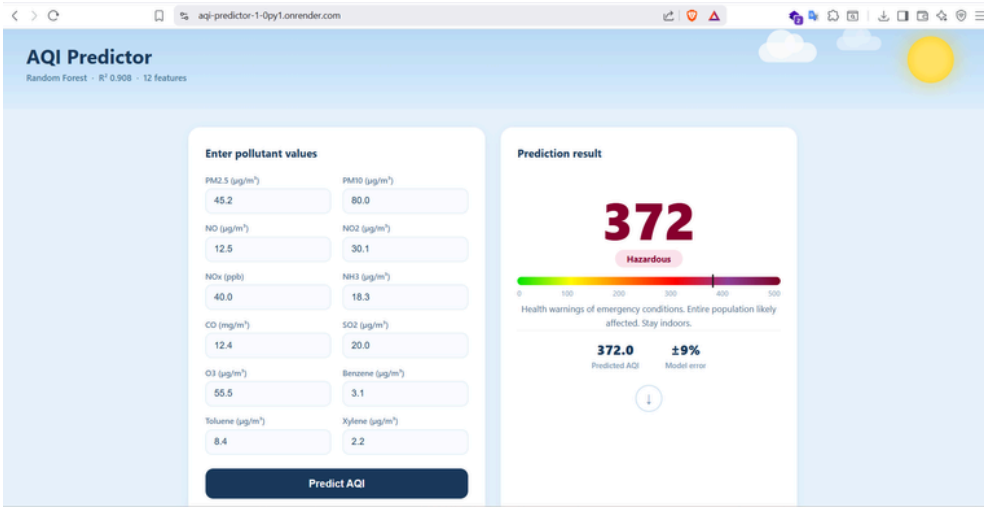




Screenshots / Results







CONCLUSION

Challenges Faced

The primary challenge was Environment Disparity, as the model file (.pkl) failed to load on the Linux-based Render cloud due to Windows-specific path issues. This was resolved by implementing a dynamic path-handling script. Additionally, Feature Alignment required strict data-mapping logic to ensure the 12 frontend inputs matched the exact column order expected by the Random Forest model to prevent inaccurate predictions.

Learnings & Key Takeaways

This project provided hands-on experience in Full-Stack ML Integration, moving beyond notebooks to a live system. Key learnings include mastering Asynchronous JavaScript (Fetch API) for seamless UI updates and understanding Production-Grade Deployment using Gunicorn and WSGI servers. It bridged the gap between theoretical data science and practical software engineering.

Future Enhancements

Future versions will focus on Real-Time Geo-Location Integration, enabling the app to automatically fetch local sensor data via API based on the user's coordinates. I also plan to implement Time-Series Forecasting using LSTM neural networks to evolve the platform from a current-state calculator into a predictive tool for 48-hour air quality trends.

References

- Dataset: Kaggle
- Web Framework: Pallets Projects - Flask Documentation
- Machine Learning: Scikit-learn - Random Forest Regressor API
- Deployment: Render Cloud - Python/Gunicorn Deployment Guide.
- Theory: Breiman, L. (2001). Random Forests. Journal of Machine Learning.